

Exercícios de autorrevisão

10.1 Preencha os espaços em cada uma das sentenças:

- a) Um(a) _____ é uma coleção de variáveis relacionadas e agrupadas sob um único nome.
- b) Um(a) _____ é uma coleção de variáveis agrupadas sob um único nome em que as variáveis compartilham o mesmo espaço de armazenamento.
- c) Os bits no resultado de uma expressão que usa o operador _____ são definidos como 1, se os bits correspondentes em cada operando forem definidos como 1. Caso contrário, os bits são definidos como zero.
- d) As variáveis declaradas em uma definição da estrutura são chamadas de _____.
- e) Em uma expressão que usa o operador _____, os bits são definidos como 1, se pelo menos um dos bits correspondentes em qualquer operando for definido como 1. Caso contrário, os bits são definidos como zero.
- f) A palavra-chave _____ introduz uma declaração da estrutura.
- g) A palavra-chave _____ é usada para criar um sinônimo para um tipo de dado previamente definido.
- h) Em uma expressão que usa o operador _____, os bits são definidos como 1, se exatamente um dos bits correspondentes em qualquer operando for definido como 1. Caso contrário, os bits são definidos como zero.
- i) O operador AND sobre bits (&) normalmente é usado para _____ bits, ou seja, selecionar certos bits e zerar outros.
- j) A palavra-chave _____ é usada para introduzir uma definição de união.
- k) O nome da estrutura é chamado de _____ da estrutura.
- l) Um membro da estrutura pode ser acessado com o operador _____ ou com _____.
- m) Os operadores _____ e _____ são usados para deslocar os bits de um valor à esquerda ou à direita, respectivamente.
- n) Um(a) _____ é um conjunto de inteiros representados por identificadores.

10.2 Responda *verdadeiro* ou *falso* para os itens a seguir. Justifique sua resposta caso haja alternativas falsas.

- a) Estruturas podem conter variáveis com apenas um tipo de dados.
- b) Duas uniões podem ser comparadas (usando ==) para que se determine se são iguais.
- c) O nome da tag de uma estrutura é opcional.
- d) Os membros de diferentes estruturas devem ter nomes exclusivos.

- e) A palavra-chave `typedef` é usada para definir novos tipos de dados.
- f) As estruturas são sempre passadas a funções por referência.
- g) As estruturas não podem ser comparadas usando os operadores `==` e `!=`.

10.3 Escreva o código para realizar cada uma das tarefas a seguir:

- a) Defina uma estrutura chamada `peça` que contenha a variável `int numPeça` e o array de `char nomePeça` com valores que tenham até 25 caracteres (incluindo o caractere nulo de finalização).
- b) Defina `Peça` para que seja um sinônimo do tipo `struct peça`.
- c) Use `Peça` para declarar que a variável `a` seja do tipo `struct peça`, o array `b[ 10 ]` seja do tipo `struct peça` e a variável `ptr` seja do tipo ponteiro para `struct peça`.
- d) Leia um número de `peça` e um nome de `peça` do teclado para os membros individuais da variável `a`.
- e) Atribua os valores membros da variável `a` ao elemento 3 do array `b`.
- f) Atribua o endereço do array `b` à variável de ponteiro `ptr`.
- g) Imprima os valores membros do elemento 3 do array `b` usando a variável `ptr` e o operador de ponteiro da estrutura para que se refira aos membros.

10.4 Encontre o erro em cada um dos itens a seguir:

- a) Suponha que `struct card` tenha sido definida para que contenha dois ponteiros para o tipo `char`, a saber, `face` e `suit`. Além disso, a variável `c` foi definida para ser do tipo `struct card` e a variável `cPtr` foi definida para ser do tipo ponteiro para `struct card`. A variável `cPtr` recebeu o endereço de `c`.

```
printf( "%s\n", *cPtr->face );
```
- b) Suponha que `struct card` tenha sido definida para que contenha dois ponteiros para o tipo `char`, a saber, `face` e `suit`. Além disso, o array `hearts[ 13 ]` foi definido para ser do tipo `struct card`. A instrução a seguir deverá imprimir o membro `face` do elemento do array 10.

```
printf( "%s\n", copas.face );
```
- c) `union` valores {

```
char w;  
float x;  
double y;  
};  
union valores v = { 1.27 };
```

```
d) struct pessoa {
    char nome[ 15 ];
    char sobrenome[ 15 ];
    int idade;
}
```

```
e) Suponha que struct pessoa tenha sido definido
    como na parte (d), mas com a correção apropriada.
        pessoa d;

f) Suponha que a variável p tenha sido declarada como
    tipo struct pessoa, e a variável c tenha sido de-
    clarada como tipo struct card.
        p = c;
```

■ Respostas dos exercícios de autorrevisão

10.1 a) estrutura. b) união. c) AND sobre bits (&). d) membros. e) OR inclusivo sobre bits (|). f) struct. g) typedef. h) OR exclusivo sobre bits (^). i) máscara. j) union. k) nome de tag. l) membro da estrutura, ponteiro da estrutura. m) operador de deslocamento à esquerda (<<), operador de deslocamento à direita (>>). n) enumeração.

10.2 a) Falso. Uma estrutura pode conter variáveis de muitos tipos de dados.

b) Falso. As uniões não podem ser comparadas porque pode haver bytes de dados indefinidos com diferentes valores nas variáveis da união, que de outra forma seriam idênticos.

c) Verdadeiro.

d) Falso. Os membros das estruturas separadas podem ter os mesmos nomes, mas os membros da mesma estrutura precisam ter nomes exclusivos.

e) Falso. A palavra-chave typedef é usada para definir novos nomes (sinônimos) para os tipos de dados previamente definidos.

f) Falso. As estruturas são sempre passadas a funções com chamadas por valor.

g) Verdadeiro, devido aos problemas de alinhamento.

```
10.3 a) struct peça {
    int numPeça;
    char nomePeça[26];
};
```

b) **typedef struct** peça Peça;

c) Peça a, b[10], *ptr;

d) scanf("%d%25s", &a.numPeça, &a.nomePeça);

e) b[3] = a;

f) ptr = b;

g) printf("%d %s\n", (ptr + 3)->numPeça, (ptr + 3)->nomePeça);

10.4 a) Os parênteses que devem delimitar *cPtr foram omitidos, fazendo com que a ordem de avaliação da expressão seja incorreta. A expressão deveria ser

```
( *cPtr )->face
```

b) O subscrito do array foi omitido. A expressão deveria ser

```
copas[ 10 ].face.
```

c) Uma união só pode ser inicializada com um valor que tenha o mesmo tipo do primeiro membro da união.

d) Um ponto e vírgula é necessário para finalizar uma definição da estrutura.

e) A palavra-chave struct foi omitida da declaração da variável. A declaração deveria ser

```
struct pessoa d;
```

f) Variáveis de diferentes tipos da estrutura não podem ser atribuídas umas às outras.

■ Exercícios

10.5 Forneça a definição de cada uma das estruturas e uniões a seguir:

a) Estrutura estoque que contém o array de caracteres nomePeça[30], o inteiro numPeça, o preço em ponto flutuante, o inteiro quantidade e o inteiro pedido.

b) Dados de união que contêm char c, short s, long b, float f e double d.

c) Uma estrutura chamada endereço, que contém os arrays de caracteres

```
rua[ 25 ], cidade[ 20 ], estado[ 2 ] e cep[ 8 ].
```

d) A estrutura aluno que contém os arrays nome [15] e sobrenome [15] e a variável end-Resid do tipo struct endereço da parte (c).

e) A estrutura teste que contém 16 campos de

bit com larguras de 1 bit. Os nomes dos campos de bit são as letras de a até p.

10.6 Dada a seguinte estrutura e definições de variável,

```
struct cliente {
    char nome[ 15 ];
    char sobrenome[ 15 ];
    int numCliente;

    struct {
        char telefone[ 11 ];
        char endereço[ 50 ];
        char cidade[ 25 ];
        char estado[ 2 ];
        char cep[ 8 ];
    } pessoal;
} regCliente, *ptrCliente;

ptrCliente = &regCliente;
```

escreva uma expressão que possa ser usada para acessar os membros da estrutura em cada uma das partes a seguir:

- a) Membro sobrenome da estrutura regCliente.
- b) Membro sobrenome da estrutura apontada por ptrCliente.
- c) Membro nome da estrutura regCliente.
- d) Membro nome da estrutura apontada por ptrCliente.
- e) Membro numCliente da estrutura regCliente.
- f) Membro numCliente da estrutura apontada por ptrCliente.
- g) Membro telefone do membro pessoal da estrutura regCliente.
- h) Membro telefone do membro pessoal da estrutura apontada por ptrCliente.
- i) Membro endereço do membro pessoal da estrutura regCliente.
- j) Membro endereço do membro pessoal da estrutura apontada por ptrCliente.
- k) Membro cidade do membro pessoal da estrutura regCliente.
- l) Membro cidade do membro pessoal da estrutura apontada por ptrCliente.
- m) Membro estado do membro pessoal da estrutura regCliente.
- n) Membro estado do membro pessoal da estrutura apontada por ptrCliente.
- o) Membro cep do membro pessoal da estrutura regCliente.
- p) Membro cep do membro pessoal da estrutura apontada por ptrCliente.

10.7 Modificação do programa de embaralhamento e distribuição de cartas. Modifique o programa da Figura 10.16 de modo que as cartas sejam embaralhadas por meio de uma técnica de alto desempenho (como mostra a Figura 10.3). Imprima o baralho resultante no formato de duas colunas, como mostra a Figura 10.4. Preceda cada carta com sua respectiva cor.

10.8 Uso de uniões. Crie a união integer com membros char c, short s, int i e long b. Escreva um programa que aceite valores do tipo char, short, int e long e armazene os valores em variáveis de união do tipo union integer. Cada variável de união deve ser impressa como um char, um short, um int e um long. Os valores sempre são impressos corretamente?

10.9 Uso de uniões. Crie a união pontoFlutuante com membros float f, double d e long double x. Escreva um programa que aceite valores do tipo float, double e long double, e armazene os valores em variáveis de união do tipo union pontoFlutuante. Cada variável de união deverá ser impressa como um float, um double e um long double. Os valores sempre são impressos corretamente?

10.10 Deslocamento de inteiros à direita. Escreva um programa que desloque para a direita uma variável inteira de 4 bits. O programa deverá imprimir o inteiro em bits antes e depois da operação de deslocamento. Seu sistema coloca 0s ou 1s nos bits vagos?

10.11 Deslocamento de inteiros à direita. Se seu computador usa inteiros de 2 bytes, modifique o programa da Figura 10.7 de modo que ele funcione com inteiros de 2 bytes.

10.12 Deslocamento de inteiros à esquerda. Deslocar à esquerda um inteiro unsigned por 1 bit é equivalente a multiplicar o valor por 2. Escreva uma função potencia2 que use dois argumentos inteiros número e pot e calcule

$$\text{número} * 2^{\text{pot}}$$

Use o operador de deslocamento para calcular o resultado. Imprima os valores como inteiros e como bits.

10.13 Compactação de caracteres em um inteiro. O operador de deslocamento à esquerda pode ser usado para compactar dois valores de caracteres em uma variável inteira unsigned. Escreva um programa que aceite dois caracteres do teclado e os passe para a função compactaCaracteres. Para compactar dois caracteres em uma variável inteira unsigned, atribua o primeiro caractere à variável unsigned, desloque a variável unsigned para a esquerda em 8 posições de bits e combine a variável unsigned com o segundo caractere

usando o operador OR inclusivo sobre bits. O programa deverá enviar os caracteres em seu formato de bit antes e depois de serem compactados no inteiro unsigned, para provar que eles foram compactados corretamente na variável unsigned.

10.14 Descompactação de caracteres a partir de um inteiro. Usando o operador de deslocamento à direita, o operador AND sobre bits e uma máscara, escreva uma função descompactaCaracteres que receba o inteiro unsigned do Exercício 10.13 e o descompacte em dois caracteres. Para descompactar dois caracteres a partir de um inteiro unsigned, combine o inteiro unsigned com a máscara 65280 (00000000 00000000 11111111 00000000) e desloque o resultado em 8 bits para a direita. Atribua o valor resultante a uma variável char. Depois, combine o inteiro unsigned com a máscara 255 (00000000 00000000 00000000 11111111). Atribua o resultado a outra variável char. O programa deverá imprimir o inteiro unsigned em bits antes que ele seja descompactado, e depois imprimir os caracteres em bits para confirmar que foram descompactados corretamente.

10.15 Compactação de caracteres em um inteiro. Se o seu sistema usa inteiros de 4 bytes, reescreva o programa do Exercício 10.13 para que ele compacte 4 caracteres.

10.16 Descompactação de caracteres a partir de um inteiro. Se seu sistema usa inteiros de 4 bytes, reescreva a função descompactaCaracteres do Exercício 10.14 para que ela descompacte 4 caracteres. Crie as máscaras necessárias para descompactar os 4 caracteres deslocando o valor 255 à esquerda em 8 bits na variável de máscara por 0, 1, 2 ou 3 vezes (a depender do byte que você estiver descompactando).

10.17 Inversão da ordem dos bits de um inteiro. Escreva um programa que inverta a ordem dos bits em um valor inteiro unsigned. O programa deverá pedir o valor do usuário e chamar a função reverseBits para imprimir os bits em ordem inversa. Imprima o valor em bits antes e depois que os bits forem invertidos, para confirmar que eles foram invertidos corretamente.

10.18 Função displayBits portátil. Modifique a função displayBits da Figura 10.7 de modo que ela seja portátil entre sistemas que usem inteiros de 2 bytes e de 4 bytes. [Dica: use o operador sizeof para determinar o tamanho de um inteiro em uma máquina específica.]

10.19 Qual é o valor de X? O programa a seguir usa a função multiple para determinar se o inteiro digitado pelo teclado é um múltiplo de um inteiro X. Examine a função multiple e depois determine o valor de X.

```

1  /* ex10_19.c */
2  /* Esse programa determina se um valor
   é um múltiplo de X. */
3  #include <stdio.h>
4
5  int multiple( int num ); /* protótipo
   */
6
7  int main( void )
8  {
9      int y; /* y terá um inteiro informado pelo usuário */
10
11     printf( "Digite um inteiro entre 1 e 32000: " );
12     scanf( "%d", &y );
13
14     /* se y é um múltiplo de X */
15     if ( multiple( y ) ) {
16         printf( "%d é um múltiplo de X\n", y );
17     } /* fim do if */
18     else {
19         printf( "%d não é um múltiplo de X\n", y );
20     } /* fim do else */
21
22     return 0; /* indica conclusão bem-sucedida */
23 } /* fim do main */
24
25 /* determina se num é um múltiplo de X */
26 int multiple( int num )
27 {
28     int i; /* contador */
29     int mask = 1; /* inicializa mask */
30     int mult = 1; /* inicializa mult */
31
32     for ( i = 1; i <= 10; i++, mask <<= 1 ) {
33
34         if ( ( num & mask ) != 0 ) {
35             mult = 0;
36             break;
37         } /* fim do if */
38     } /* fim do for */
39
40     return mult;
41 } /* fim da função multiple */

```

10.20 O que o programa a seguir faz?

```

1  /* ex10_20.c */
2  #include <stdio.h>
3
4  int mystery( unsigned bits ); /* protótipo */
5
6  int main( void )
7  {
8      unsigned x; /* x manterá um inteiro digitado pelo usuário */
9
10     printf( "Digite um inteiro: " );
11     scanf( "%u", &x );
12
13     printf( "O resultado é %d\n", mystery( x ) );
14     return 0; /* indica conclusão bem-sucedida */
15 } /* fim do main */
16
17 /* O que a função a seguir faz? */
18 int mystery( unsigned bits )
19 {
20     unsigned i; /* contador */
21     unsigned mask = 1 << 31; /* inicializa máscara */
22     unsigned total = 0; /* inicializa total */
23
24     for ( i = 1; i <= 32; i++, bits <= 1 ) {
25
26         if ( ( bits & mask ) == mask ) {
27             total++;
28         } /* fim do if */
29     } /* fim do for */
30
31     return !( total % 2 ) ? 1 : 0;
32 } /* fim da função mystery */

```

Fazendo a diferença

10.21 Informatização de registros de saúde. Ultimamente, a questão da informatização dos registros de saúde tem feito parte dos noticiários. Essa possibilidade está sendo estudada com cuidado devido a preocupações com privacidade e segurança quanto a informações confidenciais, entre outras questões. A informatização de registros de saúde poderia facilitar o compartilhamento de perfis e históricos de saúde de pacientes entre diversos médicos. Isso poderia melhorar a qualidade do atendimento, ajudaria a evitar conflitos entre medicamentos e prescrição de receitas incorretas, reduziria custos e, nas salas de emergência, salvaria vidas. Neste exercício, você criará uma estrutura `PerfilSaúde` 'inicial' para uma pessoa. Os membros da estrutura deverão incluir nome, sobrenome, sexo, data de nascimento (consistindo em atributos separados para dia, mês e ano de nascimento),

altura (em centímetros) e peso (em quilos). Seu programa deverá ter uma função que receba esses dados e os utilize para definir os membros de uma variável `PerfilSaúde`. O programa também deverá incluir funções que calculem e retornem a idade do usuário em anos, a frequência cardíaca máxima e a frequência cardíaca ideal (ver Exercício 3.48), e o índice de massa corporal (IMC; ver Exercício 2.32). O programa deverá pedir a informação da pessoa, criar uma variável `PerfilSaúde` para ela e exibir as informações dessa variável — o que inclui nome, sobrenome, sexo, data de nascimento, altura e peso; depois, deverá calcular e exibir a idade da pessoa em anos, seu IMC e suas frequências cardíacas máxima e ideal. Ele também deverá exibir a tabela de 'valores de IMC' do Exercício 2.32.